

HPC 中的计算机系统

目 录

1 引言：代码之下的隐藏世界	2
1.1 计算机系统层次栈	2
2 操作系统的双重角色	3
3 CPU 的抽象	3
4 存储层次结构	4
5 操作系统事件机制	4
5.1 陷阱：中断与异常	4
5.2 模式位：用户态与内核态	5
5.3 系统调用：进入内核的正规通道	5
5.4 定时器：夺回控制权	6
6 进程：资源分配的基本单位	6
6.1 什么是进程	6
6.2 ELF 格式：可执行文件长什么样	7
6.3 进程地址空间	7
6.4 进程调度	8
6.5 进程间通信	8
7 线程：轻量级的执行单位	9
8 虚拟内存与分页	10
8.1 为什么需要虚拟内存	10
8.2 分页机制	10
9 本章你将学会	11
10 要点速查	11
11 小结	12

1 引言：代码之下的隐藏世界

当你写下一行 C 语言代码 `int x = 1 + 2;` 时，你是否想过这行代码在硬件层面究竟发生了什么？从高级语言到晶体管开关，中间经过了层层转换。理解这些层次，是掌握 HPC 的关键起点。本章的目标，就是带你走进“代码之下的隐藏世界”，揭开从程序到硬件之间的层层抽象。

计算机系统设计的核心思想是**抽象**（Abstraction）：将具体细节泛化和剥离，只关注当前层次需要了解的重要信息。正如你开车时不需要理解发动机内部的燃烧过程，只需要知道方向盘、油门和刹车的作用。程序员不需要理解逻辑门电路就能编写程序，硬件工程师也不需要了解应用逻辑就能设计芯片。

直觉 不妨把抽象想象成一栋大楼。你住在大楼的某一层，只需要知道楼梯和电梯怎么用，不需要知道地基是怎么浇筑的、管道是怎么铺设的。计算机系统的每一层都是如此：上层只依赖下层提供的接口，不关心下层的实现细节。这种“分层不关心”的设计，让复杂的系统变得可以管理。

抽象这个概念回答了三个问题：为什么需要抽象（Why）、抽象是什么（What）、如何实现抽象（How）。带着这三个问题，我们来看计算机系统的完整层次栈。

1.1 计算机系统层次栈

计算机系统从上到下可以表示为以下层次栈，每一层都建立在其下层之上：

层次	关注的问题
Problem	如何用数学或逻辑描述问题
Algorithm	如何设计步骤解决问题
Program	如何用编程语言实现算法
Operating System	如何管理硬件资源、调度程序
ISA (指令集架构)	定义软件与硬件的接口
Microarchitecture	如何用逻辑电路实现指令
Logic Circuits	逻辑门如何组成功能电路
Devices / Circuits	晶体管和物理器件

这门课程在不同层次上有对应的课程：算法层面有数据结构与算法课，编译器层面有编译原理课，数字逻辑层面有数字逻辑设计课。而我们这门课，**Introduction to Computer Systems in HPC**，重点关注的是操作系统层和计算机体系结构层。具体来说，第一天讲操作系统（线程级并行 TLP），第二天讲计算机体系结构（数据级并行 DLP）。

其中，**ISA**（Instruction Set Architecture，指令集架构）是软硬件的交界线：编译器只需将程序翻译成 ISA 指令，CPU 设计者只需实现 ISA 定义的指令。这种分层设计使得复杂的计算机系统变得可管理、可维护，也使得软件和硬件可以独立发展。

常见的 ISA 包括 x86-64（Intel/AMD 桌面与服务器）、ARM（移动设备与苹果芯片）、RISC-V（开源指令集，广泛用于教学与研究）。本讲后续在讨论陷阱和系统调用时，将以 RISC-V 术语为例。

2 操作系统的双重角色

我们每天都在使用操作系统：Windows、macOS、iOS、Linux、Android。虽然它们看起来千差万别，但底层内核其实只有几大家族：Windows 使用 NT 内核，macOS 和 iOS 使用 Darwin 内核，Linux 和 Android 使用 Linux 内核。

抛开表面差异，操作系统的本质角色可以概括为两个：

- **资源抽象器 (Resource Abstractor)**：将复杂的物理硬件抽象为简单统一的接口。例如，无论底层是机械硬盘还是 SSD，应用程序看到的都是“文件”接口；无论物理内存如何分布，程序看到的都是连续的虚拟地址空间。
- **资源分配器 (Resource Allocator)**：当多个程序竞争 CPU、内存等资源时，操作系统决定谁获得什么资源、获得多少、何时获得。例如，当多个进程同时请求 CPU 时，调度器决定执行顺序和时间片分配。

直觉 把操作系统想象成一个“大楼管理员”。大楼里有会议室（CPU）、办公位（内存）、仓库（磁盘）、电梯和空调（设备）。管理员负责：谁什么时候用会议室（CPU 调度），谁坐哪个办公位（内存管理），东西存在仓库哪里（文件系统），怎么操作电梯和空调（设备驱动）。住户（用户程序）只需要告诉管理员需求，不需要自己去管理这些资源。

这两个角色贯穿了本章的全部内容：CPU 的抽象是“资源抽象器”的体现，进程调度和内存管理是“资源分配器”的体现，而陷阱、系统调用和定时器，则是操作系统实现这两个角色的技术手段。

3 CPU 的抽象

如果我们把目光聚焦到 CPU 芯片内部，比如一张 Intel Core i9 13900K 的 die shot（芯片布局图），你会看到密密麻麻的计算单元、缓存、总线。但对于操作系统来说，不需要关心这些物理细节，CPU 被抽象为两个核心要素：

- **程序计数器 (Program Counter, PC)**：指向下一条要执行的指令地址。CPU 的工作就是不断重复“取指令、分析指令、执行指令”的循环，PC 就是这个循环的“指针”，告诉 CPU 下一步该从哪里取指令。
- **寄存器与上下文 (Context)**：保存当前执行状态的所有寄存器值。寄存器是 CPU 内部最快的存储单元，存储着当前正在处理的数据。

直觉 为什么只需要 PC 和寄存器就能抽象整个 CPU？因为 CPU 执行程序的本质就是“取指-执行”的循环。只要我们知道“下一条指令在哪”（PC）和“当前的数据状态是什么”（寄存器），就能完整描述 CPU 在某一时刻的执行状态。这就像你在读一本书，只要记住“读到第几页”和“当前在读什么内容”，就能随时暂停并继续。

通过保存和恢复 PC 和上下文，操作系统可以在多个程序之间快速切换 CPU，造成“多个程序同时运行”的假象，这就是**多道程序设计 (Multiprogramming)**的基础。这种切换称为**上下文切换 (Context Switch)**，其开销主要来自寄存器的保存与恢复以及缓存失效。

4 存储层次结构

计算机的存储系统是一个多层次的金字塔结构，从快到慢、从小到大：

层次	速度	容量
寄存器 Register	最快	最小
L1 / L2 / L3 缓存	很快	较小
主存 DRAM	中等	较大
SSD / 磁盘	较慢	大
网络存储	最慢	最大

这种层次结构利用了**局部性原理**（Principle of Locality）：程序倾向于反复访问近期访问过的数据（时间局部性，Temporal Locality）和地址空间相近的数据（空间局部性，Spatial Locality）。

直觉 不妨想想你办公桌上的东西。你正在写的文件放在手边（寄存器），最近用过的参考书放在抽屉里（缓存），书架上的书要起身去拿（主存），图书馆的书要出门去找（磁盘）。你不会把所有书都摊在桌上，因为桌面空间有限。同样，CPU 不会把所有数据都放在寄存器里，而是根据“最近会用”的预测，把热点数据放在离自己最近的缓存层。

缓存命中时数据极快，缓存未命中时需要从更慢的层级加载。在 HPC 优化中，理解存储层次并优化数据访问模式（如缓存分块 blocking）是提升性能的关键手段。我们会在后续章节深入讨论缓存友好的编程技巧。

5 操作系统事件机制

操作系统如何知道“有事发生了”？答案是**事件驱动**。操作系统的核心可以理解为一个不断处理事件（陷阱）的循环。让我们先从最基础的“什么是事件”开始。

5.1 陷阱：中断与异常

在 RISC-V 术语中，**陷阱**（Trap）是一个统称，包含两种情况：

- **中断**（Interrupt）：由外部硬件异步生成。例如键盘输入、定时器到期、网卡收到数据。中断是“硬件主动告诉 CPU：有事情需要你处理”。
- **异常**（Exception）：由程序运行时同步生成。例如除零错误、缺页错误、非法指令。异常是“程序自己出了问题，CPU 被迫停下来处理”。

两者的关键区别在于“谁来触发”：中断来自外部硬件，与当前执行的指令无关；异常来自当前指令的执行过程，是同步发生的。

RISC-V 官方手册原文是：“Exceptions, Traps, and Interrupts”。在 RISC-V 中，Trap 是 Interrupt 和 Exception 的统称。不同体系结构的术语可能略有不同，例如 x86 中用 “Interrupt” 泛指两者。

当陷阱发生时，CPU 暂停当前执行，保存 PC 和上下文，跳转到操作系统预设的**陷阱处理程序**（Trap Handler）。处理完成后，操作系统决定恢复原程序执行还是终止进程。这种控制流的切换称为**受困控制流**（Trapped Control Flow）。

5.2 模式位：用户态与内核态

陷阱发生时，控制流会从用户空间切换到内核空间。那么，操作系统如何区分“当前是在运行用户程序还是在运行内核代码”呢？答案是引入一个**模式位**（Mode Bit）：

- **用户态**（User Mode）：应用程序在此模式下运行，权限受限，不能直接访问硬件和特权内存。
- **内核态**（Kernel Mode）：操作系统在此模式下运行，拥有全部硬件访问权限。

直觉 模式位就像一栋大楼的门禁卡。普通住户（用户程序）只有普通卡，能进自己的房间和公共区域，但不能进机房和配电室。管理员（操作系统内核）有万能卡，哪里都能进。当住户需要维修服务时，必须通过前台（系统调用）让管理员代为处理，而不是自己闯进机房。

模式位是一个标志，区分当前处于哪种模式。这种设计确保了应用程序的错误不会破坏整个系统：一个进程崩溃不会导致其他进程或操作系统本身受损。

5.3 系统调用：进入内核的正规通道

当用户程序需要做特权操作时，比如读写文件、分配内存、发送网络数据，它不能直接操作硬件，而必须通过**系统调用**（System Call）请求操作系统代为执行。系统调用是一种特殊的陷阱。

系统调用的过程为：

1. 应用程序将参数放入寄存器
2. 执行特殊指令（如 RISC-V 的 `ecall`）主动触发陷阱
3. 硬件切换到内核态并跳转到陷阱入口
4. 操作系统根据系统调用号分发到对应处理函数
5. 执行完毕后返回用户态，将结果交给应用程序

例 假设你的程序调用 `read(fd, buf, 100)` 读取文件。这条 C 库函数最终会执行 `ecall` 指令，把系统调用号（代表 `read`）和参数（文件描述符 `fd`、缓冲区地址 `buf`、字节数 `100`）放入寄存器。CPU 触发陷阱后切到内核态，操作系统的系统调用处理函数根据调用号找到 `sys_read`，代你完成磁盘读取，然后把读到的数据写进 `buf`，最后返回用户态并把实际读取的字节数返回给程序。

在操作系统的内核代码中，所有事件的处理都可以抽象为一个 `processEvent` 函数：

```
void processEvent(event) {
    switch (event.type) {
        case NETWORK_COMMUNICATION:
            NetworkManager.handleEvent(event);
            break;
        case SEGMENTATION_FAULT:
```

```

case INVALID_MODE:
    ProcessManager.handleEvent(event);
    break;
case SYSTEM_CALL:
    SystemCallManager.handleEvent(event);
    break;
// ...
}
}

```

这段伪代码展示了内核的**事件循环**（Event Loop）本质：等待事件 → 根据事件类型分发到对应的处理器 → 处理完后继续等待。段错误导致进程被终止，系统调用根据调用号分发到文件、内存或网络等子系统。

5.4 定时器：夺回控制权

有一个问题：如果用户程序进入了死循环，操作系统还有机会运行吗？如果 CPU 完全被用户程序占用，操作系统就无法调度其他进程，整个系统可能被“锁死”。

解决方案是**硬件定时器**（Timer）。操作系统在启动时设置一个定时器，让它定期产生中断。定时器中断也是一种陷阱，触发时控制权交还给操作系统，操作系统借此机会检查“当前进程的时间片是否用完”，如果用完就切换到下一个进程。

直觉 定时器就像一个闹钟。操作系统给每个进程分配一段“使用时间”（时间片），同时设好闹钟。闹钟一响，操作系统就回来检查：这个进程是不是用太久了？如果是，就换一个进程上来。这样即使某个进程陷入死循环，也不会永久霸占 CPU，因为闹钟会定期把控制权还给操作系统。

定时器中断是实现**抢占式调度**（Preemptive Scheduling）的基础。内核的事件循环中，定时器中断对应 `TIMER_INTERRUPT` 分支，由调度器重新评估进程优先级并决定下一个运行的进程。

6 进程：资源分配的基本单位

前面我们了解了操作系统如何管理 CPU 和响应事件，现在来看操作系统管理的核心对象：进程。

6.1 什么是进程

进程（Process）是正在执行的程序，是操作系统进行资源分配和保护的基本单位。每个进程拥有独立的地址空间和系统资源（如文件描述符表、信号处理表）。进程之间相互隔离，一个进程的错误不会直接影响其他进程。

直觉 进程和程序的区别，就像“正在播放的电影”和“硬盘上的电影文件”的区别。程序是静态的，存在磁盘上；进程是动态的，加载到内存中运行。同一个程序可以同时运行多个进程（比如同时打开两个浏览器窗口），每个进程有独立的内存空间和状态。

6.2 ELF 格式：可执行文件长什么样

Linux 下的可执行文件采用 **ELF**（Executable and Linkable Format，可执行与可链接格式）格式。让我们编译一个简单的程序并查看文件格式：

```
gcc hello.c -o hello
./hello
file hello
```

逐行解释：

- `gcc hello.c -o hello`：调用 GCC 编译器，将 C 源文件 `hello.c` 编译为可执行文件 `hello`。
- `./hello`：运行该程序，输出 `Hello World!`。
- `file hello`：使用 `file` 命令查看文件类型。

`file` 命令的输出类似：

```
hello: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
for GNU/Linux 3.2.0, not stripped
```

这告诉我们：`hello` 是一个 64 位的 ELF 可执行文件，采用小端字节序（LSB），动态链接，解释器为 `/lib64/ld-linux-x86-64.so.2`。ELF 文件按 **section**（段）组织，包含代码段（`.text`）、数据段（`.data`）、BSS 段（`.bss`）、符号表、调试信息等。ELF 格式不仅用于可执行文件，还用于目标文件（`.o`）和动态链接库（`.so`）。

6.3 进程地址空间

当 ELF 文件被加载到内存成为进程后，操作系统为它创建一个独立的虚拟地址空间，包含以下区域：

区域	内容
Code (Text)	编译后的机器指令，只读且可共享
Data	已初始化的全局变量
BSS	未初始化的全局变量，加载时清零
Heap	动态分配的内存（ <code>malloc / new</code> ）
Stack	函数调用帧、局部变量
Context	保存的寄存器和执行状态（PC 等）

其中 Code 段最初存储在磁盘上的 ELF 文件中，加载时映射到内存。Stack 向低地址增长，Heap 向高地址增长，两者之间是空闲的地址空间，按需扩展。Context 保存了进程的执行状态，包括 PC 和寄存器值，正是上一节“CPU 抽象”中提到的两个核心要素。

Stack 中存放的是栈帧（Stack Frame）。每次函数调用都会在栈上压入一个栈帧，包含返回地址、参数、局部变量和保存的寄存器。函数返回时弹出栈帧，恢复调用者的状态。理解栈帧对调试程序崩溃（如段错误）至关重要。

例 C 程序的内存布局

考虑以下 C 代码中变量的内存分布：

```
int x;
int y = 15;
int *values;
values = malloc(100 * sizeof(int));
```

逐个分析：

- x 未初始化，是全局变量，位于 **BSS 段**（未初始化数据区，加载时自动清零）。
- y 已初始化为 15，是全局变量，位于 **Data 段**（已初始化数据区）。
- values 是局部变量（指针本身），位于 **Stack**。
- malloc 分配的 100 个整数的空间位于 **Heap**（动态分配区）。

取一个小数值验证：假设 int 占 4 字节，malloc(100 * 4) 会在 Heap 上分配 400 字节。y = 15 的值 15 直接存储在 Data 段中（占用 4 字节），而 x 虽然也占 4 字节，但它在 BSS 段中只是占位符，加载时被清零，不占用 ELF 文件空间。

理解变量的存储位置对调试内存错误和优化缓存利用至关重要。例如，频繁访问的数组如果在 Heap 上，可能产生缓存未命中；如果能改为栈上分配或使用 register 提示，可能提升性能。

6.4 进程调度

操作系统通过**调度器**（Scheduler）决定哪个进程在何时使用 CPU。调度策略包括：

- **非抢占式**（Non-preemptive）：进程主动让出 CPU，操作系统不强制切换。进程必须完成执行后，OS 才能调度下一个进程。
- **抢占式**（Preemptive）：操作系统通过定时器中断强制切换进程，无需等待进程主动让出。
- **Round Robin**（轮转调度）：每个进程分配固定时间片，时间片用完后排到队尾，循环执行。
- **Priority-based**（优先级调度）：高优先级进程优先执行。
- **Core Affinity**（核心亲和性）：将进程绑定到特定 CPU 核心，减少缓存失效和上下文切换开销。

直觉 Round Robin 就像食堂排队打饭：每个人固定打几分钟，时间到了就排到队尾，下一个人上。Priority-based 则像 VIP 通道：紧急任务（高优先级）可以插队。Core Affinity 则像固定窗口：一个人总是去同一个窗口打饭，师傅记住他的口味（缓存热度），打饭更快。

调度器不仅要决定“谁先运行”，还要决定“在哪个核心上运行”。这就是核心亲和性的作用：将进程绑定到特定 CPU 核心，避免进程在核心间迁移导致缓存失效。在 HPC 中，MPI 进程通常通过 Slurm 等调度器分配到特定节点和核心，核心亲和性设置确保进程不会在核心间迁移，从而保持缓存热度。

6.5 进程间通信

进程之间地址空间隔离，无法直接访问彼此的内存。当多个进程需要协作时，就需要**进程间通信**（Inter-Process Communication, IPC）机制：

- **Message Passing** (消息传递): 通过系统调用在进程间传递数据, 如 `msgsnd/msgrcv`。MPI 就是一种消息传递机制。
- **Shared Memory** (共享内存): 多个进程映射同一块物理内存, 是最快的 IPC 方式, 但需要同步。
- **Signal** (信号): 如 `SIGKILL` (强制终止)、`SIGTERM` (请求终止)、`SIGINT` (中断)。
- **Pipe** (管道): 父子进程间单向通信, 分为匿名管道和命名管道 (FIFO)。
- **Socket** (套接字): 支持跨主机网络通信, 是 TCP/IP 的基础。

IPC 虽然实现了进程间协作, 但开销不小。由于进程间地址空间隔离, 消息传递需要维护消息队列, 并且数据需要在地址空间之间拷贝; 共享内存虽然不需要拷贝, 但需要同步机制, 且在进程切换时必须刷新缓存 (因为不同进程的虚拟地址映射到不同的物理页)。这就是“IPC 太重”的原因, 也是引入线程的动机。

7 线程: 轻量级的执行单位

IPC 太重了, 有没有更轻量化的方式让多个执行流共享数据? 答案是**线程** (Thread), 也称为**轻量级进程** (Light-Weight Process, LWP)。线程是进程内的基本执行单位, 一个进程可以包含多个线程:

- **线程独有**: Thread ID、程序计数器、寄存器集合、栈
- **线程共享**: 代码段、数据段、堆、文件描述符、信号处理表

直觉 进程和线程的关系, 就像“房子”和“住的人”。进程是一栋独立的房子, 有自己的厨房 (堆)、书房 (代码段)、仓库 (数据段)。线程是住在同一栋房子里的人, 每人有自己的卧室 (栈和寄存器), 但共用厨房、书房和仓库。因为共用空间, 人与人之间交流 (通信) 很方便, 但也要注意别同时抢同一个锅 (数据竞争)。

线程间共享内存使得通信比 IPC 高效得多: 不需要系统调用, 不需要数据拷贝, 直接读写共享变量即可。但这也带来了**数据竞争** (Data Race) 的风险: 当多个线程同时读写同一变量时, 结果取决于执行顺序, 导致不可预测的行为。

例 假设两个线程同时对共享变量 `balance = 100` 执行 `balance = balance + 50`。预期结果是 200, 但如果两个线程同时读到 `balance = 100`, 各自加 50 后写回 150, 最终结果是 150 而非 200, 丢失了一次更新。这就是数据竞争的典型表现。

因此需要**锁** (Mutex, 互斥量)、**条件变量** (Condition Variable) 等同步机制来保护共享数据。**OpenMP** (开放式多处理) 正是在线程模型之上提供了简化的并行编程接口, 让程序员不必手动管理线程和锁, 后续章节会详细讲解。

线程的“并发” (Concurrency) 能力是多线程的核心价值: 一个多线程进程可以同时做多件事, 比如一个线程负责计算, 另一个线程负责 I/O。但在单核 CPU 上, 这种“同时”是通过快速切换实现的假象; 只有多核 CPU 才能真正同时执行多个线程。

8 虚拟内存与分页

最后一个核心概念是**虚拟内存**（Virtual Memory）。它是进程和物理内存之间的又一层抽象。

8.1 为什么需要虚拟内存

如果没有虚拟内存，每个进程直接操作物理内存，会带来严重问题：进程 A 可以读写进程 B 的内存，一个进程的错误可能破坏整个系统。虚拟内存为每个进程提供独立的、连续的地址空间，带来三大好处：

- **隔离**（Isolation）：每个进程拥有自己的地址空间，一个进程无法访问另一个进程的内存。
- **保护**（Protection）：通过页表权限位控制内存访问，防止越权读写。操作系统可以拒绝进程访问其他进程或特权内存。
- **效率**（Efficiency）：支持**写时复制**（Copy-on-Write, CoW）等技术，高效利用物理内存；还能运行部分加载的程序，不需要把整个程序一次性载入内存。

直觉 写时复制的典型场景是 `fork()`。父进程调用 `fork()` 创建子进程时，子进程并不立即复制父进程的全部内存，而是与父进程共享物理页，只把页表标记为“只读”。当某一方尝试写入时，硬件触发异常，操作系统才真正复制那一页。这样，如果子进程只是读取数据而不修改，就完全不需要复制内存，大大节省了时间和空间。

8.2 分页机制

虚拟内存如何映射到物理内存？最常用的实现方式是**分页**（Paging）。分页机制涉及以下核心概念：

- **Page**（页）：虚拟内存的最小管理单元，通常大小为 4 KB。虚拟内存被划分为固定大小的页。
- **Frame**（帧）：物理内存中与页大小相同的块。物理内存被划分为固定大小的帧。
- **MMU**（Memory Management Unit，内存管理单元）：硬件部件，负责将虚拟地址翻译为物理地址。
- **Page Table**（页表）：记录虚拟页到物理帧映射关系的数据结构，通常采用多级页表以节省内存。
- **Page Fault**（缺页异常）：当 CPU 访问的页不在物理内存中时触发，操作系统从磁盘加载该页并更新页表。

例 假设页大小为 4 KB (2^{12} 字节)，虚拟地址为 32 位。那么虚拟地址被分为两部分：高 20 位是**虚拟页号**（Virtual Page Number, VPN），低 12 位是**页内偏移**（Offset）。MMU 查页表，找到虚拟页号 0x00314 对应的物理帧号 0x00821。最终物理地址 = 物理帧号 × 页大小 + 页内偏移。

取一个小数值：虚拟地址 0x00314008，高 20 位页号 0x00314，低 12 位偏移 0x008。查页表得到帧号 0x00821，则物理地址为 0x00821008。偏移部分保持不变，只是页号变成了帧号。

地址翻译过程为：CPU 生成虚拟地址 → MMU 查页表获取物理帧号 → 组合帧号和页内偏移得到物理地址 → 访问物理内存。

为加速翻译，CPU 内置了 **TLB** (*Translation Lookaside Buffer*，页表缓存)，缓存最近的页表项。在 HPC 中，大页 (*HugePages*，如 2 MB 或 1 GB) 可以减少 TLB 未命中，提升大规模数据访问的性能。后续实验中我们会实际配置 *HugePages* 来加速矩阵运算。

9 本章你将学会

1. 理解抽象作为计算机系统设计的核心思想，能够描述从 Problem 到 Circuits 的八层次栈及 ISA 作为软硬件界面的作用。
2. 能够解释操作系统的双重角色（资源抽象器与资源分配器），理解 CPU 被抽象为 PC 和上下文的原因。
3. 区分中断与异常（陷阱的两种形式），理解模式位、系统调用和定时器如何共同实现用户态与内核态的隔离与切换。
4. 能够分析一个 C 程序中各变量在进程地址空间（Code / Data / BSS / Heap / Stack）中的位置，并解释 ELF 格式的意义。
5. 理解进程与线程的区别（资源分配单位 vs 执行单位，隔离 vs 共享），能够解释数据竞争的成因和虚拟内存分页机制的工作原理。

10 要点速查

概念	要点
Abstraction	将具体细节泛化剥离，只关注当前层次重要信息
层次栈	Problem → Algorithm → Program → OS → ISA → Microarchitecture → Logic → Circuits
ISA	指令集架构，软硬件交界线，如 x86-64 / ARM / RISC-V
OS 双角色	资源抽象器（统一接口）+ 资源分配器（决定谁获得资源）
CPU 抽象	PC（下一条指令地址）+ Context（寄存器状态）
存储层次	寄存器 → Cache → DRAM → SSD/磁盘 → 网络存储，速度递减容量递增
局部性	时间局部性（重复访问）+ 空间局部性（地址相近）
Trap	陷阱 = 中断（外部异步）+ 异常（内部同步）
Mode Bit	用户态（受限）vs 内核态（全权限）
System Call	用户态进入内核态的通道，通过 <code>ecall</code> 触发
Timer	定期中断，确保 OS 能夺回 CPU 控制权，实现抢占式调度
Process	程序在执行中，资源分配和保护的基本单位
ELF	可执行与可链接格式，按 section 组织
地址空间	Code / Data / BSS / Heap / Stack / Context
调度策略	非抢占式 / 抢占式 / Round Robin / Priority / Core Affinity
IPC	Message Passing / Shared Memory / Signal / Pipe / Socket
Thread	轻量级进程，共享代码/数据/堆/文件，独有 PC/寄存器/栈

数据竞争	多线程同时读写共享变量导致不可预测行为，需同步机制
虚拟内存	隔离 + 保护 + 效率 (CoW)，进程与物理内存间的抽象层
Paging	Page (虚拟页) + Frame (物理帧) + MMU + Page Table + Page Fault
TLB	页表缓存，加速虚拟地址到物理地址的翻译

11 小结

本讲我们从“代码之下的隐藏世界”出发，首先理解了抽象作为计算机系统设计的核心思想，以及从 Problem 到 Circuits 的八层层次栈，其中 ISA 是软硬件的交界线。随后我们深入操作系统，理解了它的双重角色：资源抽象器（将硬件抽象为统一接口）和资源分配器（决定资源的分配）。

在 CPU 抽象部分，我们学会了用 PC 和 Context 两个要素描述 CPU 的执行状态，这是上下文切换的基础。存储层次结构让我们理解了金字塔形的存储系统和局部性原理。操作系统的事件机制，包括陷阱（中断与异常）、模式位、系统调用和定时器，共同构成了操作系统管理硬件和响应用户请求的核心机制。

进程是资源分配的基本单位，我们分析了 ELF 格式、进程地址空间的六大区域，以及 C 程序中变量在内存中的分布。线程是进程内的执行单位，共享内存带来高效通信的同时也引入了数据竞争的风险。虚拟内存通过分页机制实现了进程间的隔离与保护，MMU 和页表完成了虚拟地址到物理地址的翻译。

这些概念是后续学习并行编程的基础：理解线程才能学习 OpenMP，理解进程和 IPC 才能学习 MPI，理解存储层次和虚拟内存才能进行有效的性能优化。下一讲我们将进入计算机体系结构的世界，学习如何利用硬件的并行能力实现数据级并行。

讲义基于 HPC101 2025 Day1「Introduction to Computer Systems in HPC」课程内容
编写